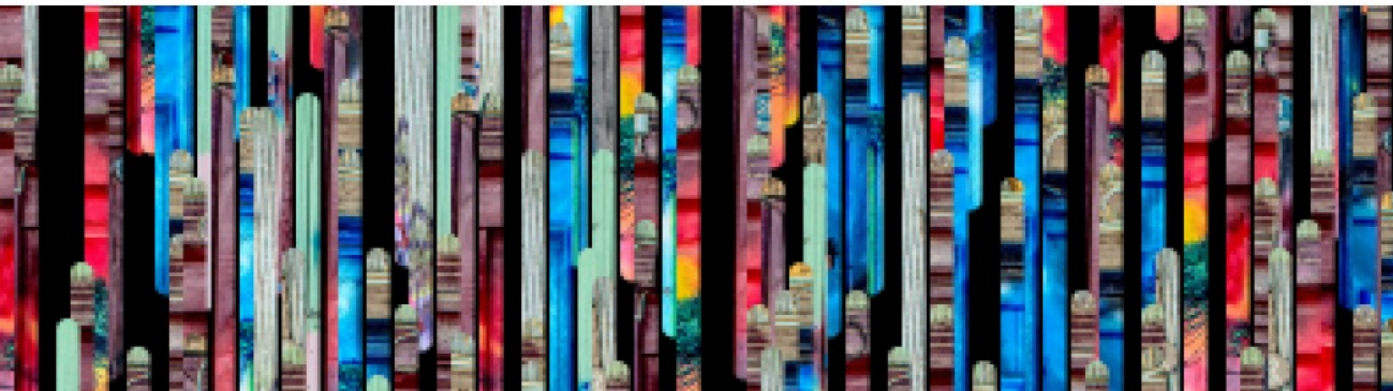
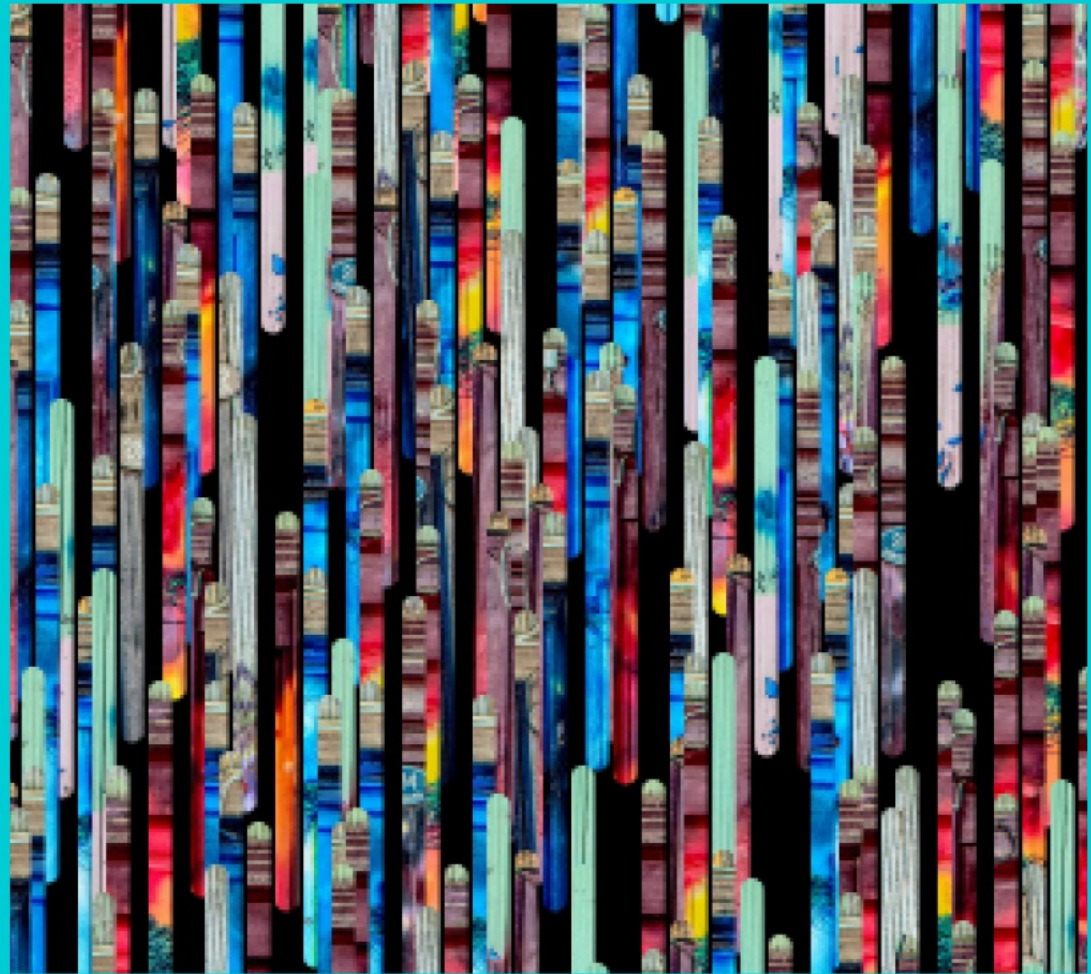




SQL FOR DATA ANALYSTS · LECTURE 2

DDL & DML, Procedures, Variables

Denis Boldin





Contents

- 2.1 Creating and updating tables
- 2.2 Materializations: views and materialized views
- 2.3 Procedures
- 2.4 Variables
- Summary

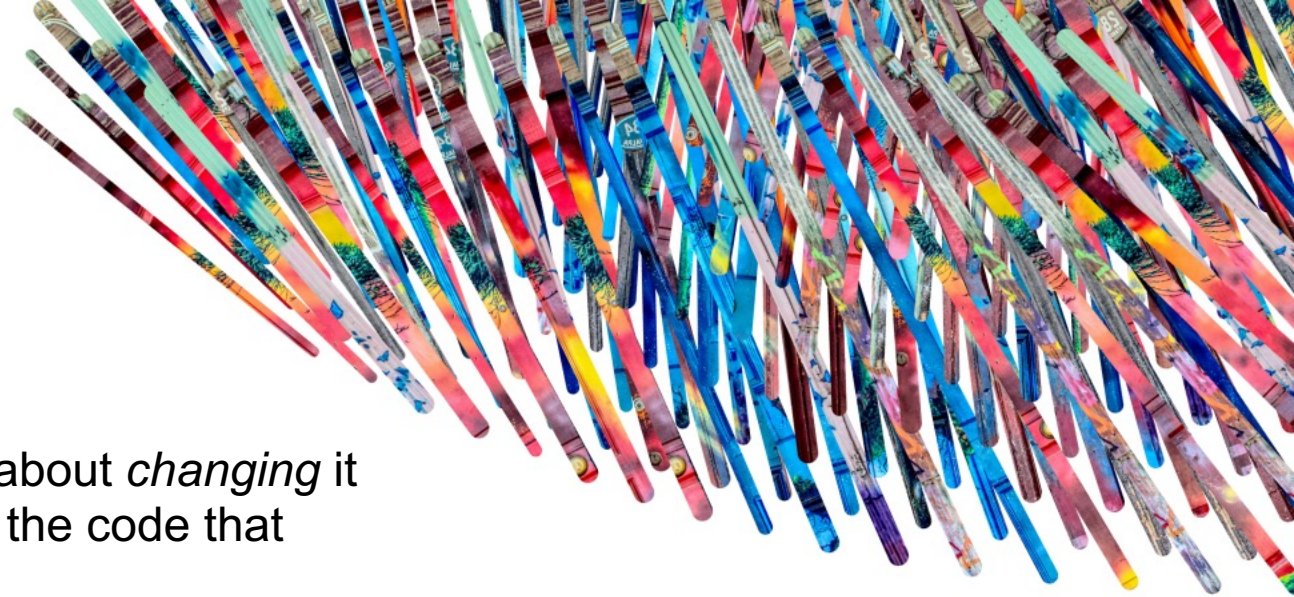


Contents

Lecture 1 was about *reading* data. This lecture is about *changing* it — the structure of the database, its contents, and the code that lives inside the database itself.

Everything runs against the same course database, in the [live playground](#). This time you will be modifying it, which is exactly why the playground is worth having: **refresh the page and you get a clean database back**. Nothing you do here can break anything.

Rather than mutate `tracks` and `artists`, most of this lecture builds something new on top of them — a `playlists` table, the feature every music service has and this dataset lacks.

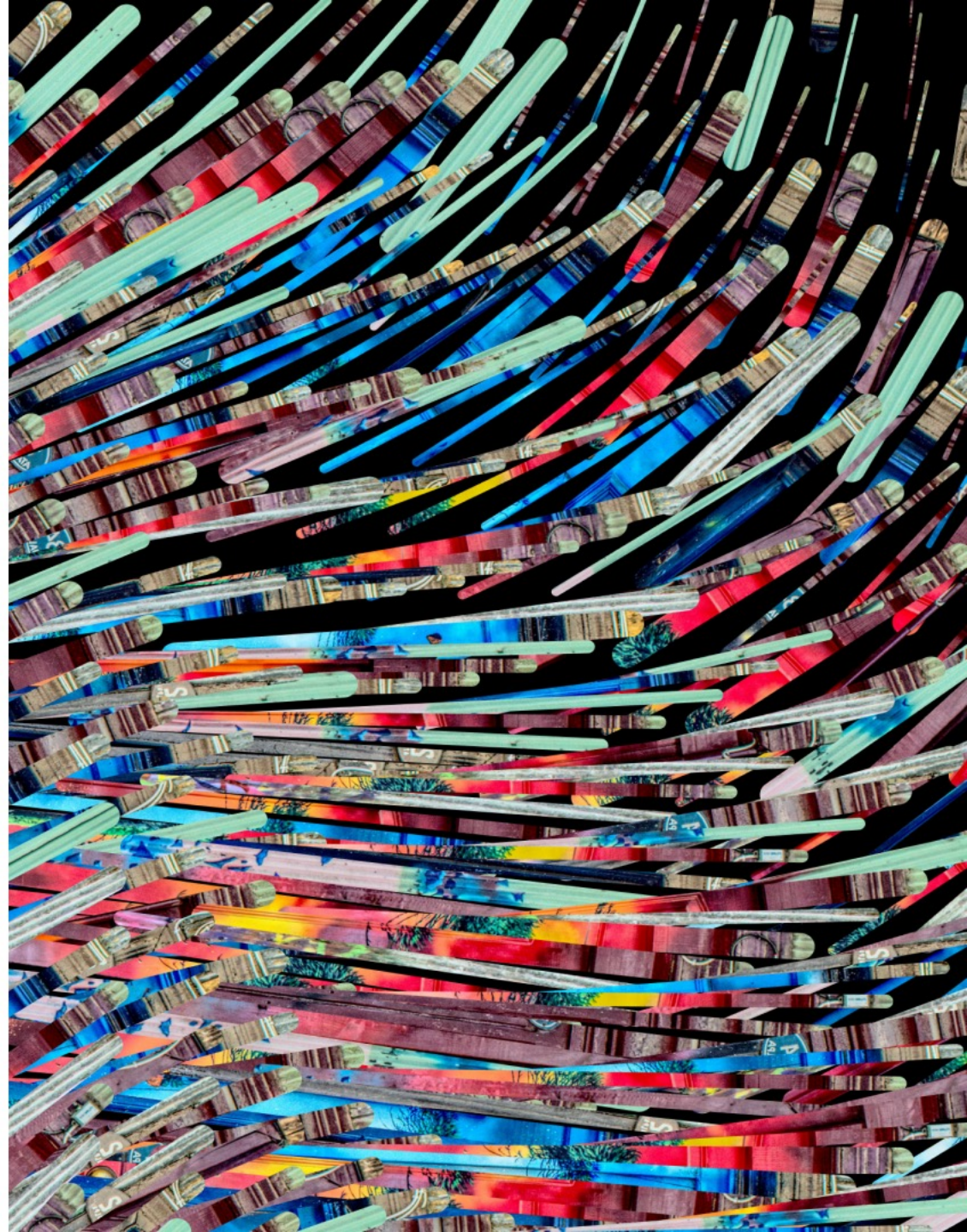




2.1 Creating and updating tables

SQL statements are traditionally split into groups by what they act on:

- CREATE
- ALTER
- TRUNCATE
- DROP
- INSERT
- UPDATE
- DELETE
- DELETE vs TRUNCATE
- A note on safety





2.1 Creating and updating tables

Group	Statements	Acts on
DDL — Data Definition Language	<code>CREATE</code> , <code>ALTER</code> , <code>DROP</code> , <code>TRUNCATE</code>	the structure: tables, indexes, constraints, sequences
DML — Data Manipulation Language	<code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code>	the data inside the tables
TCL — Transaction Control Language	<code>BEGIN</code> , <code>COMMIT</code> , <code>ROLLBACK</code>	transactions (see ACID, Lecture 1 §1.6)
DCL — Data Control Language	<code>GRANT</code> , <code>REVOKE</code>	access rights

This section covers the first two.



CREATE

Creating a table in its simplest form:

```
CREATE TABLE playlists (  
    id          int,  
    name       text,  
    owner      text,  
    created_at  date  
);
```



CREATE

That works, but it says nothing about what is *valid*. In practice you also declare keys, length limits and constraints. The same table, written the way you would actually write it in PostgreSQL:

```
CREATE TABLE playlists (  
  id          SERIAL          PRIMARY KEY,  
  name        VARCHAR(120) NOT NULL,  
  owner       VARCHAR(80)  NOT NULL,  
  is_public   BOOLEAN       NOT NULL DEFAULT true,  
  created_at  TIMESTAMP     NOT NULL DEFAULT now(),  
  CONSTRAINT playlists_name_per_owner UNIQUE (name, owner)  
);  
  
INSERT INTO playlists (name, owner) VALUES  
  ('Late Night Drive', 'denis'),  
  ('Loud and Fast',    'denis'),  
  ('Quiet Mornings',  'anna');  
  
SELECT id, name, owner, is_public FROM playlists ORDER BY id;
```

RESULT

id	name	owner	is_public
1	Late Night Drive	denis	true
2	Loud and Fast	denis	true
3	Quiet Mornings	anna	true



CREATE

- `id` is not a plain `INT` but a `SERIAL PRIMARY KEY` — an auto-incrementing surrogate key, which is why the `INSERT` never mentions `id`;
- `name` is a `VARCHAR` limited to 120 characters instead of an unbounded `TEXT`;
- `owner` carries a `NOT NULL` constraint;
- `is_public` and `created_at` have `DEFAULT`s, so the `INSERT` did not have to supply them;
- the table-level `UNIQUE` constraint spans **two** columns: one owner may not have two playlists with the same name, but two owners may.



CREATE

💡 DDL syntax differs considerably between DBMSs — ClickHouse, for example, additionally requires a table engine, a sort key and `ON CLUSTER` for a distributed rollout. Always consult the documentation of the system you are working with. Everything in this course is PostgreSQL.

Now the table that connects playlists to the catalogue — a bridge table, exactly like `track_artists` from Lecture 1:

```
CREATE TABLE playlist_tracks (  
    playlist_id INT NOT NULL REFERENCES playlists(id) ON DELETE CASCADE,  
    track_id TEXT NOT NULL REFERENCES tracks(id),  
    position SMALLINT NOT NULL,  
    added_at TIMESTAMP NOT NULL DEFAULT now(),  
    PRIMARY KEY (playlist_id, track_id)  
);  
  
SELECT count(*) AS rows_so_far FROM playlist_tracks;
```

RESULT

rows_so_far
0



CREATE

- **PRIMARY KEY** uniquely identifies a row; there is exactly one per table, and it is **NOT NULL** by definition. Here it spans two columns — the same track cannot be added to the same playlist twice.
- **REFERENCES** (a foreign key) guarantees that the value exists in the referenced table — this is the "Consistency" of ACID being enforced for you.
- **ON DELETE CASCADE** says what to do when the parent goes: delete a playlist and its entries go with it. Without it, PostgreSQL would refuse the delete.
- **UNIQUE** forbids repeated values.
- **DEFAULT** supplies a value when the **INSERT** does not.



CREATE

The foreign key is not decoration. Try to add a track that does not exist and the database refuses:

```
INSERT INTO playlist_tracks (playlist_id, track_id, position)
VALUES (1, 'no-such-track-id', 1);
-- ERROR: insert or update on table "playlist_tracks" violates foreign key
-- constraint "playlist_tracks_track_id_fkey"
```



ALTER

ALTER TABLE renames tables and adds, modifies and removes fields.

Renaming a table:

```
ALTER TABLE playlists RENAME TO user_playlists;  
  
SELECT id, name, owner FROM user_playlists ORDER BY id;
```

RESULT

id	name	owner
1	Late Night Drive	denis
2	Loud and Fast	denis
3	Quiet Mornings	anna

Renaming is usually fast: no new table is created and no data is copied — one name is simply replaced by another in the DBMS's catalogue.



ALTER

-- put the name back, so the rest of the lecture keeps working

```
ALTER TABLE user_playlists RENAME TO playlists;
```

```
SELECT count(*) AS playlists FROM playlists;
```

RESULT

playlists
3



ALTER

Adding a new field. In all existing rows the new field takes the value `NULL`:

```
ALTER TABLE playlists ADD COLUMN description text;  
  
SELECT id, name, description FROM playlists ORDER BY id;
```

RESULT

id	name	description
1	Late Night Drive	<i>NULL</i>
2	Loud and Fast	<i>NULL</i>
3	Quiet Mornings	<i>NULL</i>



ALTER

Renaming a field:

```
ALTER TABLE playlists RENAME COLUMN description TO blurb;  
  
SELECT id, name, blurb FROM playlists ORDER BY id;
```

RESULT

id	name	blurb
1	Late Night Drive	<i>NULL</i>
2	Loud and Fast	<i>NULL</i>
3	Quiet Mornings	<i>NULL</i>



ALTER

Setting a default value for a field:

```
ALTER TABLE playlists ALTER COLUMN blurb SET DEFAULT '(no description)';

INSERT INTO playlists (name, owner) VALUES ('Added Later', 'denis');

SELECT id, name, blurb FROM playlists ORDER BY id;
```

RESULT

id	name	blurb
1	Late Night Drive	<i>NULL</i>
2	Loud and Fast	<i>NULL</i>
3	Quiet Mornings	<i>NULL</i>
4	Added Later	(no description)



ALTER

💡 The default value applies only to new rows. Rows added earlier keep their NULL — look at the first three rows above.

Changing a column's type. `USING` tells PostgreSQL how to convert the existing values:

```
ALTER TABLE playlists ADD COLUMN rating SMALLINT;
```

```
ALTER TABLE playlists  
ALTER COLUMN rating TYPE NUMERIC(3,1)  
USING rating::numeric;
```

```
SELECT column_name, data_type, numeric_precision, numeric_scale  
FROM information_schema.columns  
WHERE table_name = 'playlists' AND column_name = 'rating';
```

RESULT

column_name	data_type	numeric_precision	numeric_scale
rating	numeric	3	1



ALTER

`information_schema` is how you ask PostgreSQL about its own structure — useful whenever you are not sure what a table actually looks like.





TRUNCATE

TRUNCATE TABLE removes all rows very quickly — much faster than **DELETE FROM table**, because it does not delete rows one by one. The table structure stays in place.

```
CREATE TABLE playlists_copy AS SELECT * FROM playlists;  
  
SELECT count(*) AS before_truncate FROM playlists_copy;
```

RESULT

before_truncate
4



TRUNCATE

```
TRUNCATE TABLE playlists_copy;  
  
SELECT count(*) AS after_truncate FROM playlists_copy;
```

RESULT

after_truncate
0

`CREATE TABLE ... AS SELECT` above is worth noting in its own right: it creates a table and fills it from a query in one statement, inferring the column types.



DROP

Dropping a column:

```
ALTER TABLE playlists DROP COLUMN rating;
```

```
SELECT column_name  
  FROM information_schema.columns  
 WHERE table_name = 'playlists'  
 ORDER BY ordinal_position;
```

RESULT

column_name
id
name
owner
is_public
created_at
blurb



DROP

Dropping a table:

```
DROP TABLE playlists_copy;  
  
SELECT count(*) AS matching_tables  
  FROM information_schema.tables  
 WHERE table_name = 'playlists_copy';
```

RESULT

matching_tables
0

💡 Be careful with dropping a table — restoring it from a backup will be very hard. `DROP TABLE IF EXISTS` avoids an error when the table may not be there, which is what you want in a script; it does not make the drop any less permanent.



INSERT

Adding records from literal values — we did this above:

```
INSERT INTO playlists (name, owner) VALUES  
  ('Late Night Drive', 'denis'),  
  ('Loud and Fast',    'denis');
```



INSERT

Adding rows **from a query** is where it gets interesting. Filling a playlist from the catalogue — energetic tracks at a running tempo, most popular first:

```
INSERT INTO playlist_tracks (playlist_id, track_id, position)
SELECT 1,
       t.id,
       ROW_NUMBER() OVER (ORDER BY t.popularity DESC, t.name)
FROM tracks AS t
WHERE t.tempo BETWEEN 150 AND 170
      AND t.energy > 0.8
      AND t.popularity > 60
ORDER BY t.popularity DESC, t.name
LIMIT 10;

SELECT pt.position, t.name, t.popularity, round(t.tempo::numeric, 0) AS bpm
FROM playlist_tracks AS pt
JOIN tracks AS t ON t.id = pt.track_id
WHERE pt.playlist_id = 1
ORDER BY pt.position;
```



INSERT

RESULT

position	name	popularity	bpm
1	Amor da Despedida	83	168
2	Último Beijo	83	157
3	Happy - From "Despicable Me 2"	81	160
4	Everlong	79	158
5	Giants	77	158
6	Live Your Life	77	160
7	Meia Noite (Você tem meu Whatsapp)	77	150
8	Pump It	77	154
9	Basta Você Me Ligar - Ao Vivo	75	168
10	Crazy	75	162



INSERT

The fields selected in `SELECT` must match, in order and in type, the fields listed for the target table. If a field has no source, put something explicit — a constant, or `NULL` — in its place, as we did with the literal `1` for `playlist_id`.

`ON CONFLICT` handles the case where the row is already there. Without it, re-running the insert above would fail on the primary key:

```
INSERT INTO playlists (id, name, owner)
VALUES (1, 'Late Night Drive (renamed)', 'denis')
ON CONFLICT (id) DO UPDATE
    SET name = EXCLUDED.name;

SELECT id, name, owner FROM playlists ORDER BY id;
```

RESULT

id	name	owner
1	Late Night Drive (renamed)	denis
2	Loud and Fast	denis
3	Quiet Mornings	anna
4	Added Later	denis

`EXCLUDED` is the row that *would* have been inserted. This "insert, or update if it exists" pattern is called an **upsert**, and it is everywhere in data loading.



UPDATE

Changing data in all records of a column:

```
UPDATE playlists SET is_public = false;  
  
SELECT id, name, is_public FROM playlists ORDER BY id;
```

RESULT

id	name	is_public
1	Late Night Drive (renamed)	false
2	Loud and Fast	false
3	Quiet Mornings	false
4	Added Later	false



UPDATE

Changing data in filtered records:

```
UPDATE playlists
  SET is_public = true,
      blurb      = 'Curated by ' || owner
 WHERE owner = 'denis';

SELECT id, name, owner, is_public, blurb FROM playlists ORDER BY id;
```

RESULT

id	name	owner	is_public	blurb
1	Late Night Drive (renamed)	denis	true	Curated by denis
2	Loud and Fast	denis	true	Curated by denis
3	Quiet Mornings	anna	false	<i>NULL</i>
4	Added Later	denis	true	Curated by denis



UPDATE

An `UPDATE` can also draw its new values from another table. Storing each playlist's average track popularity:

```
ALTER TABLE playlists ADD COLUMN avg_popularity numeric(5,2);

UPDATE playlists AS p
  SET avg_popularity = stats.avg_pop
  FROM (
    SELECT pt.playlist_id, round(avg(t.popularity), 2) AS avg_pop
      FROM playlist_tracks AS pt
      JOIN tracks AS t ON t.id = pt.track_id
      GROUP BY pt.playlist_id
    ) AS stats
 WHERE p.id = stats.playlist_id;

SELECT id, name, avg_popularity FROM playlists ORDER BY id;
```

RESULT

id	name	avg_popularity
1	Late Night Drive (renamed)	78.40
2	Loud and Fast	NULL
3	Quiet Mornings	NULL
4	Added Later	NULL

`UPDATE ... FROM` is PostgreSQL's way of writing "update these rows using a join". Playlists with no tracks keep `NULL`, which is correct: the average of nothing is not zero.



DELETE

```
DELETE FROM playlists WHERE owner = 'anna';
```

```
SELECT id, name, owner FROM playlists ORDER BY id;
```

RESULT

id	name	owner
1	Late Night Drive (renamed)	denis
2	Loud and Fast	denis
4	Added Later	denis



DELETE

💡 `DELETE` should always have a condition. If you mean to delete everything, `TRUNCATE` is the right statement.

`DELETE ... RETURNING` gives back the rows it removed — useful when you want to log or archive them:

```
DELETE FROM playlist_tracks
WHERE playlist_id = 1
      AND position > 8
RETURNING playlist_id, track_id, position;
```

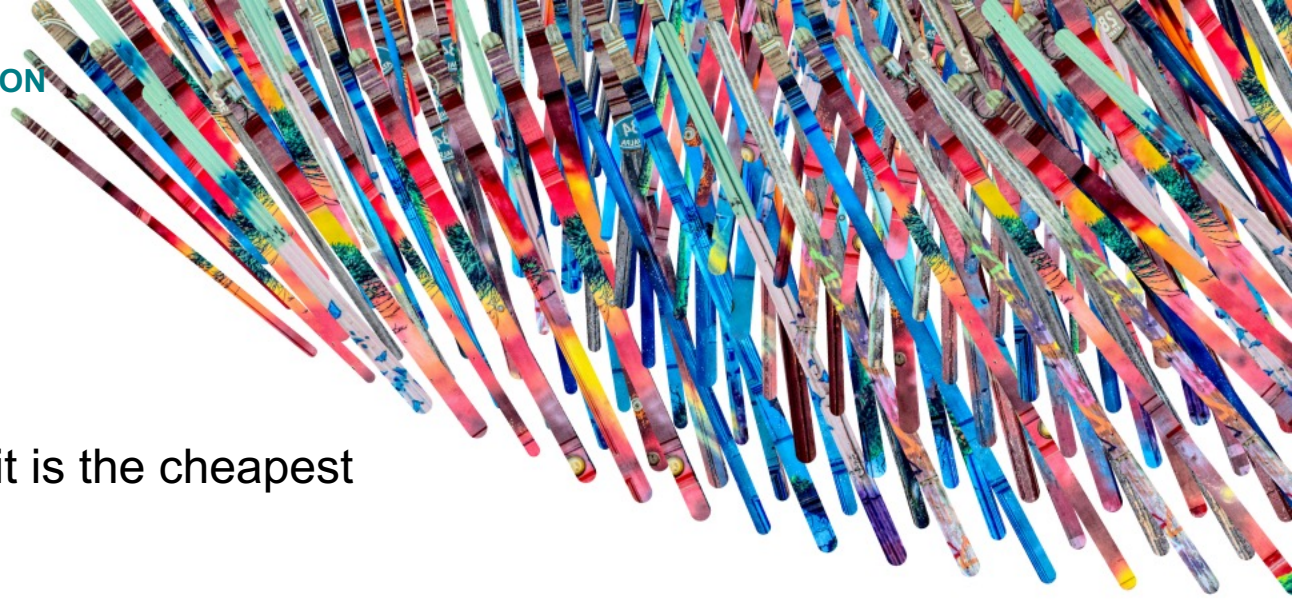
RESULT

playlist_id	track_id	position
1	3rRin3LyLY92kpEbkCgwf4	9
1	74irxdVWstNIEQjsvArITq	10



DELETE

`RETURNING` works on `INSERT` and `UPDATE` too, and it is the cheapest way to find out what a statement actually did.





DELETE vs TRUNCATE

- **DELETE** removes rows one by one and is normally used with a filter condition. It can clear a whole table, but slowly, since every row is processed separately. It can be rolled back.
- **TRUNCATE** clears the whole table at once. It is much faster, because no row-by-row deletion happens — at the cost of not being able to delete selectively.





A note on safety

Every DML statement in this section changes data. Two habits worth forming now:

1. Write the **SELECT** first, check what it returns, and only then convert it into a **DELETE** or an **UPDATE** with the same **WHERE**.
2. Wrap risky changes in a transaction, so a mistake can be undone:

```
BEGIN;
```

```
UPDATE playlists SET owner = 'oops';
```

```
SELECT id, name, owner FROM playlists ORDER BY id;    -- check the result
```

```
ROLLBACK;                                           -- or COMMIT if it looks right
```

```
SELECT id, name, owner FROM playlists ORDER BY id;    -- unchanged
```



A note on safety

RESULT

id	name	owner
1	Late Night Drive (renamed)	denis
2	Loud and Fast	denis
4	Added Later	denis

Note the missing `WHERE` on that `UPDATE` — the classic accident. Inside a transaction it costs nothing.



2.2 Materializations: views and materialized views

When several source tables are joined, filtered and aggregated, and the result is needed again and again, writing the query out repeatedly becomes laborious. Instead, give the query a name.

- Views
- Materialized views
- Check yourself



Views

A **view** ([VIEW](#)) is a named query. You address it as if it were a table — but it is not a table: the data stays in the source tables, and every time the view is addressed the same query is executed anew.

Lecture 1 kept re-writing the three-table join between tracks and artists. A view retires that repetition for good:

```
CREATE VIEW v_tracks_with_artists AS
SELECT t.id,
       t.name      AS track,
       t.popularity,
       t.release_date,
       string_agg(a.name, ', ' ORDER BY ta.position) AS artists
FROM tracks AS t
JOIN track_artists AS ta ON ta.track_id = t.id
JOIN artists AS a      ON a.id = ta.artist_id
GROUP BY t.id, t.name, t.popularity, t.release_date;

SELECT track, artists, popularity
FROM v_tracks_with_artists
ORDER BY popularity DESC
LIMIT 8;
```



Views

RESULT

track	artists	popularity
Peaches (feat. Daniel Caesar & Giveon)	Justin Bieber, Daniel Caesar, Giveon	100
Blinding Lights	The Weeknd	96
WITHOUT YOU	The Kid LAROI	94
LA NOCHE DE ANOCHE	Bad Bunny, ROSALÍA	93
DÁKITI	Bad Bunny, Jhay Cortez	92
What You Know Bout Love	Pop Smoke	91
BICHOTA	KAROL G	91
Anyone	Justin Bieber	90



Views

Now the view can be filtered, joined and aggregated like any table:

```
SELECT (EXTRACT(YEAR FROM release_date)::int / 10) * 10 AS decade,  
       count(*) AS collaborations  
FROM v_tracks_with_artists  
WHERE artists LIKE '%,%'           -- more than one credited artist  
GROUP BY decade  
ORDER BY decade;
```

RESULT

decade	collaborations
1920	213
1930	436
1940	806
1950	1221
1960	804
1970	743
1980	831
1990	1015
2000	1070
2010	2296
2020	627



Views

Replacing and dropping a view:

```
CREATE OR REPLACE VIEW v_tracks_with_artists AS
SELECT ...;           -- the new definition

DROP VIEW v_tracks_with_artists;
```

`CREATE OR REPLACE` can add columns to the end of a view but cannot rename or remove existing ones — for that you must `DROP` and re-create.



Views

Advantages:

- simplifies work with frequently used queries — a view can be reused in different queries;
- improves readability and is convenient when providing data to stakeholders;
- restricts access — users can be granted access to particular views that expose only the data they are allowed to see, without exposing the whole database.

💡 Data is not stored in a view. Every time it is addressed, the source tables are joined, filtered and aggregated again. Using a view does not save computing resources.

View vs CTE:

- A view is a permanent object in the database and can be used by *other* queries, by other people, tomorrow. A CTE lives within a single query and disappears with it.
- A CTE can be part of a view. That matters, for example, for recursive queries, which are impossible without a CTE.



Materialized views

A **materialized view** looks like an ordinary one, but the result of the query is saved into a physical table stored on disk. That is the trade-off: reads become cheap, but the data is a snapshot and can go stale.

```
CREATE VIEW v_decade_counts AS
SELECT (EXTRACT(YEAR FROM release_date)::int / 10) * 10 AS decade,
       count(*) AS tracks
FROM tracks
GROUP BY 1;

CREATE MATERIALIZED VIEW mv_decade_counts AS
SELECT (EXTRACT(YEAR FROM release_date)::int / 10) * 10 AS decade,
       count(*) AS tracks
FROM tracks
GROUP BY 1;

SELECT 'view' AS source, tracks FROM v_decade_counts WHERE decade = 2020
UNION ALL
SELECT 'materialized',    tracks FROM mv_decade_counts WHERE decade = 2020;
```

RESULT

source	tracks
view	2068
materialized	2068



Materialized views

Identical, so far. Now add a track and ask both again — **without** refreshing:

```
INSERT INTO tracks (id, name, popularity, duration_ms, explicit,
                    release_date, release_date_precision)
VALUES ('demo0000000000000000000001', 'A Brand New Song', 0, 180000, false,
        DATE '2021-06-01', 'day');

SELECT 'view' AS source, tracks FROM v_decade_counts WHERE decade = 2020
UNION ALL
SELECT 'materialized',    tracks FROM mv_decade_counts WHERE decade = 2020;
```

RESULT

source	tracks
view	2069
materialized	2068



Materialized views

There it is: the view re-ran the query and saw the new row; the materialized view returned its stored snapshot and did not. That is the whole concept in one result.





Materialized views

```
REFRESH MATERIALIZED VIEW mv_decade_counts;
```

```
SELECT 'view' AS source, tracks FROM v_decade_counts WHERE decade = 2020
```

```
UNION ALL
```

```
SELECT 'materialized', tracks FROM mv_decade_counts WHERE decade = 2020;
```

RESULT

source	tracks
view	2069
materialized	2069



Materialized views

In PostgreSQL the refresh is explicit — `REFRESH MATERIALIZED VIEW`, run manually or on a schedule. Other DBMSs differ substantially: some refresh in real time, some by a trigger, and some have no such functionality at all. Always consult the documentation.

A materialized view can be indexed, which an ordinary view cannot:

```
CREATE UNIQUE INDEX mv_decade_counts_decade ON mv_decade_counts(decade);

SELECT indexname, indexdef
  FROM pg_indexes
 WHERE tablename = 'mv_decade_counts';
```

RESULT

indexname	indexdef
mv_decade_counts_decade	CREATE UNIQUE INDEX mv_decade_counts_decade ON public.mv_decade_counts USING btree (decade)



Materialized views

A unique index also unlocks **REFRESH MATERIALIZED VIEW CONCURRENTLY**, which rebuilds it without locking readers out.





Materialized views

	View	Materialized view	Table
Stores data	no	yes	yes
Cost of reading	the full query, every time	a read of stored rows	a read of stored rows
Freshness	always current	as of the last refresh	always current
Occupies disk	no (only the definition)	yes	yes
Can be indexed	no	yes	yes



Materialized views

```
-- tidy up the demo row  
DELETE FROM tracks WHERE id = 'demo0000000000000000000001';  
REFRESH MATERIALIZED VIEW mv_decade_counts;  
  
SELECT tracks FROM mv_decade_counts WHERE decade = 2020;
```

RESULT

tracks

2068



Check yourself

What are views used for? — to simplify collecting data from the same source tables and reduce duplicated code.

How does a materialized view differ from an ordinary one? — a materialized view stores the result physically and must be refreshed; an ordinary view is simply a named query, always current.

When is a materialized view the wrong choice? — when the data must always be current, or when it changes far more often than it is read.

What happens if the schema of the table a view is built on changes? — the view breaks; drop and re-create it.

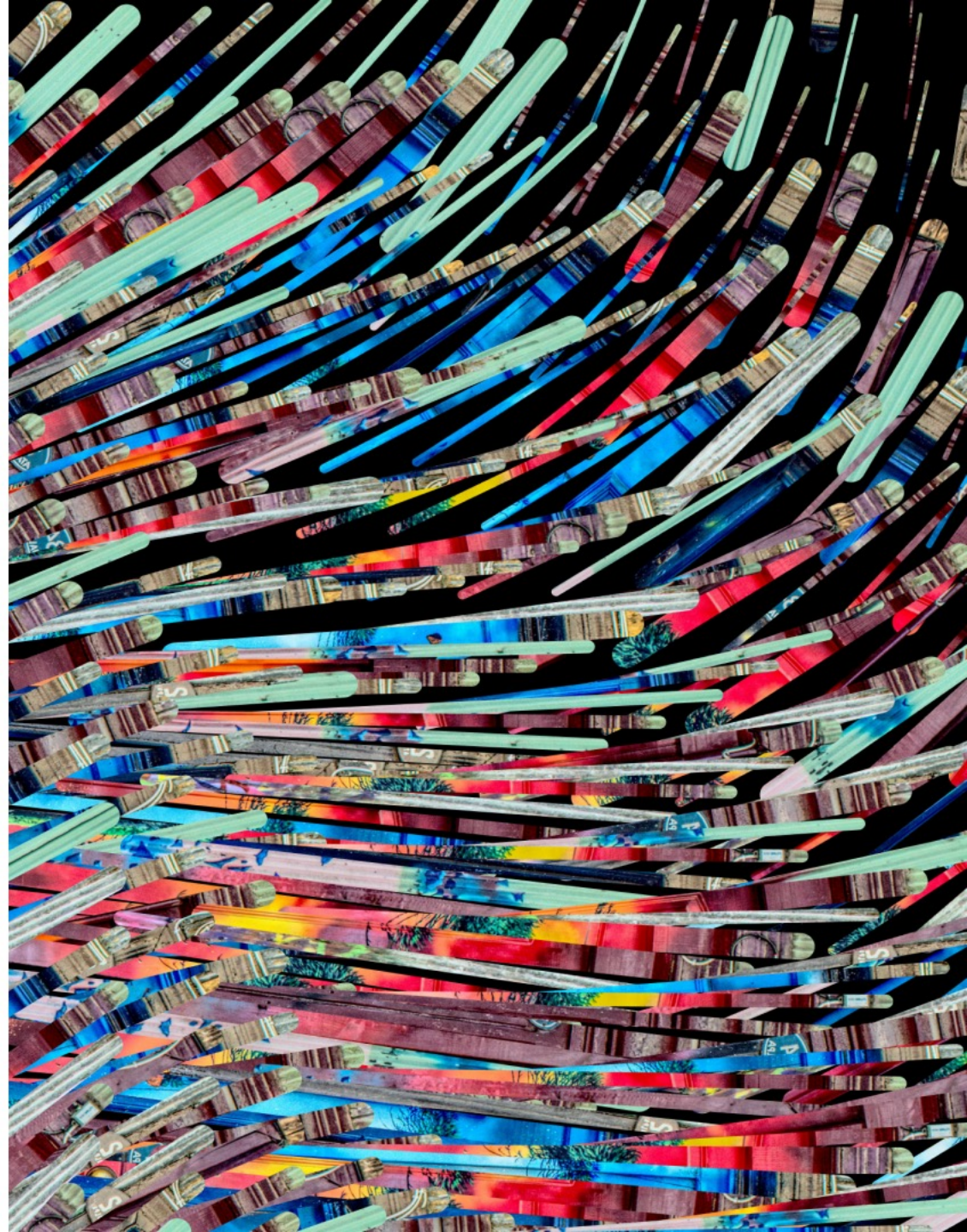




2.3 Procedures

Views name a *query*. Procedures and functions name a *sequence of statements* — logic that lives inside the database and can contain variables, conditions and loops. In PostgreSQL this code is written in **PL/pgSQL**.

- Function or procedure?
- A function
- A procedure
- Parameters
- Control flow
- Error handling
- Dropping a routine





Function or procedure?

PostgreSQL has both, and the difference matters:

	FUNCTION	PROCEDURE
Called with	<code>SELECT my_func(...)</code>	<code>CALL my_proc(...)</code>
Returns	a value or a set of rows	nothing (only <code>INOUT</code> parameters)
Usable inside a query	yes	no
Can manage transactions	no	yes (<code>COMMIT</code> / <code>ROLLBACK</code> inside)

Rule of thumb: if you need a **value**, write a function; if you need an **action**, write a procedure.



A function

The decade expression has appeared in nearly every query so far. It belongs in a function:

```
CREATE OR REPLACE FUNCTION decade_of(d date)
RETURNS int
LANGUAGE sql
IMMUTABLE
AS $$
    SELECT (EXTRACT(YEAR FROM d)::int / 10) * 10;
$$;

SELECT decade_of(release_date) AS decade, count(*) AS tracks
FROM tracks
GROUP BY 1
ORDER BY 1;
```

RESULT

decade	tracks
1900	1
1920	778
1930	1333
1940	1845
1950	3617
1960	4834
1970	6325
1980	8419
1990	11135
2000	8881
2010	10764
2020	2068



2.3 PROCEDURES

A function

That one was `LANGUAGE sql` — a single expression, no procedural logic needed. `IMMUTABLE` promises the same input always gives the same output, which lets the planner cache and even index it.

For anything with variables or branching, use `LANGUAGE plpgsql`:

```
CREATE OR REPLACE FUNCTION track_count(artist_name text)
RETURNS integer
LANGUAGE plpgsql
AS $$
DECLARE
    result integer;
BEGIN
    SELECT count(*)
    INTO result
    FROM artists AS a
    JOIN track_artists AS ta ON ta.artist_id = a.id
    WHERE a.name = artist_name;
```

```
    RETURN result;
END;
$$;

SELECT track_count('David Bowie') AS bowie,
       track_count('Bob Dylan')   AS dylan,
       track_count('Nobody')      AS nobody;
```

RESULT

bowie	dylan	nobody
32	42	0



2.3 PROCEDURES

A function

Because a function returns a value, it can be used anywhere a value is expected — including inside another query:

```
SELECT name, track_count(name) AS tracks
FROM artists
ORDER BY followers DESC
LIMIT 5;
```

RESULT

name	tracks
Ed Sheeran	10
Ariana Grande	14
Drake	32
Justin Bieber	37
Eminem	29



A function

💡 That query is elegant and slow: the function runs once per row, each time executing its own join. A single `GROUP BY` would do the same work in one pass. Convenience and performance pull in opposite directions here — Lecture 3 returns to the theme.





A function

Note the structure, which is the same for every PL/pgSQL routine:

- `$$ ... $$` — **dollar quoting**. The body is a string; dollar quoting saves you from escaping every quote inside it.
- `DECLARE` — the block where local variables are declared.
- `BEGIN ... END;` — the executable block. (This `BEGIN` is a block delimiter, not the transaction `BEGIN` from §2.1.)





2.3 PROCEDURES

A procedure

A procedure performs an action. Rebuilding a summary table:

```
CREATE TABLE decade_summary (  
    decade          int PRIMARY KEY,  
    tracks          int NOT NULL,  
    avg_popularity  numeric(5,2)  
);  
  
CREATE OR REPLACE PROCEDURE rebuild_decade_summary()  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    affected integer;  
BEGIN  
    DELETE FROM decade_summary;
```

```
INSERT INTO decade_summary (decade, tracks, avg_popularity)  
SELECT decade_of(release_date),  
       count(*),  
       round(avg(popularity), 2)  
FROM tracks  
GROUP BY 1;  
  
GET DIAGNOSTICS affected = ROW_COUNT;  
  
RAISE NOTICE 'decade_summary rebuilt with % rows', affected;  
END;  
$$;  
  
CALL rebuild_decade_summary();  
  
SELECT * FROM decade_summary ORDER BY decade;
```



A procedure

RESULT

decade	tracks	avg_popularity
1900	1	19.00
1920	778	1.05
1930	1333	2.00
1940	1845	1.76
1950	3617	8.32
1960	4834	18.18
1970	6325	24.19
1980	8419	25.94
1990	11135	29.35
2000	8881	36.60
2010	10764	39.29
2020	2068	41.12



A procedure

`GET DIAGNOSTICS ... = ROW_COUNT` reads how many rows the previous statement touched; `RAISE NOTICE` prints a message to the client — the closest thing PL/pgSQL has to `print`. The playground shows notices above the result grid.





2.3 PROCEDURES

Parameters

Parameters have three modes:

```
-- a working copy, so the demo leaves the catalogue alone
CREATE TABLE tracks_pool AS
SELECT id, name, popularity FROM tracks;

CREATE OR REPLACE PROCEDURE archive_unpopular(
    IN    threshold integer DEFAULT 5,
    INOUT moved      integer DEFAULT 0
)
LANGUAGE plpgsql
AS $$
BEGIN
    CREATE TABLE IF NOT EXISTS tracks_archive (LIKE tracks_pool);
```

```
WITH moved_rows AS (
    DELETE FROM tracks_pool
    WHERE popularity < threshold
    RETURNING *
)
INSERT INTO tracks_archive SELECT * FROM moved_rows;

GET DIAGNOSTICS moved = ROW_COUNT;
END;
$$;

CALL archive_unpopular(1, NULL);
```



2.3 PROCEDURES

Parameters

RESULT

moved

4575



2.3 PROCEDURES

Parameters

- **IN** (the default) — an input parameter;
- **OUT** — an output parameter;
- **INOUT** — both; this is how a procedure returns anything at all.

CALL on a procedure with an **INOUT** parameter returns a row, which is why the result above has a column. Check what it did:

```
SELECT (SELECT count(*) FROM tracks_pool) AS left_in_pool,  
       (SELECT count(*) FROM tracks_archive) AS archived,  
       (SELECT count(*) FROM tracks) AS catalogue_untouched;
```

RESULT

left_in_pool	archived	catalogue_untouched
55425	4575	60000



Parameters

`DELETE ... RETURNING` feeding an `INSERT` through a CTE is the idiomatic way to move rows between tables in one statement — atomic, and a single pass.





Control flow

Conditions:

```
IF track_cnt > 100 THEN
    RAISE NOTICE 'prolific';
ELSIF track_cnt > 10 THEN
    RAISE NOTICE 'established';
ELSE
    RAISE NOTICE 'occasional';
END IF;
```



Control flow

Loops:

-- a counted loop

```
FOR i IN 1..5 LOOP  
    RAISE NOTICE 'decade %', 1960 + i * 10;  
END LOOP;
```

-- a loop over the rows of a query

```
FOR rec IN SELECT name, popularity FROM tracks LIMIT 10 LOOP  
    RAISE NOTICE '% scores %', rec.name, rec.popularity;  
END LOOP;
```

-- a conditional loop

```
WHILE cnt < 10 LOOP  
    cnt := cnt + 1;  
END LOOP;
```



2.3 PROCEDURES

Control flow

All three, in a routine that classifies artists:

```
CREATE OR REPLACE FUNCTION artist_tier(artist_name text)
RETURNS text
LANGUAGE plpgsql
AS $$
DECLARE
    cnt integer;
BEGIN
    cnt := track_count(artist_name);

    IF cnt = 0 THEN
        RETURN 'unknown artist';
    ELSIF cnt > 100 THEN
        RETURN 'prolific';
    ELSIF cnt > 10 THEN
        RETURN 'established';
    ELSE
        RETURN 'occasional';
    END IF;
END;
$$;

SELECT name, track_count(name) AS tracks, artist_tier(name) AS tier
FROM artists
WHERE name IN ('David Bowie', 'Bob Dylan', 'Johann Sebastian Bach', 'Nobody At All')
ORDER BY tracks DESC;
```



2.3 PROCEDURES

Control flow

RESULT

name	tracks	tier
Johann Sebastian Bach	195	prolific
Bob Dylan	42	established
David Bowie	32	established

💡 Row-by-row loops are the slowest way to work with a relational database. If the same result can be expressed as a single set-based `UPDATE` or `INSERT ... SELECT`, write that instead — the loop is for the cases where it genuinely cannot.



Error handling

An `EXCEPTION` block catches errors so that a routine can continue instead of aborting the whole transaction:

```
CREATE OR REPLACE FUNCTION safe_add_playlist(p_name text, p_owner text)
RETURNS text
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO playlists (name, owner) VALUES (p_name, p_owner);
    RETURN 'created';
EXCEPTION
    WHEN unique_violation THEN
        RETURN 'already exists, skipped';
    WHEN others THEN
        RETURN 'unexpected error: ' || SQLERRM;
END;
$$;

SELECT safe_add_playlist('Brand New Mix', 'denis') AS first_call,
       safe_add_playlist('Brand New Mix', 'denis') AS second_call;
```

RESULT

first_call	second_call
created	already exists, skipped



Error handling

The second call hit the `UNIQUE (name, owner)` constraint from §2.1 and was caught instead of blowing up. `SQLERRM` holds the error message, `SQLSTATE` the error code.





Dropping a routine

```
DROP FUNCTION safe_add_playlist(text, text);
```

```
SELECT count(*) AS still_there  
  FROM information_schema.routines  
 WHERE routine_name = 'safe_add_playlist';
```

RESULT

still_there
0

The argument types are part of the identity of a routine — PostgreSQL allows overloading, so **DROP** needs the signature, not just the name.



2.4 Variables

There are two quite different things called "variables" in PostgreSQL. Do not mix them up.

- 1. Variables inside PL/pgSQL
- An anonymous block: DO
- 2. Session variables
- 3. psql client variables
- Which one to use



1. Variables inside PL/pgSQL

Declared in the **DECLARE** block of a function, procedure or anonymous block, and living only for the duration of the call.

DECLARE

track_cnt	integer;	<i>-- no initial value → NULL</i>
threshold	integer := 10;	<i>-- with an initial value</i>
label	text := 'prolific';	
decade	constant int := 1970;	<i>-- cannot be reassigned</i>
rec	record;	<i>-- a row of any shape</i>
t	tracks%ROWTYPE;	<i>-- a row of the tracks table</i>
nm	tracks.name%TYPE;	<i>-- the type of one column</i>

BEGIN

...

END;



1. Variables inside PL/pgSQL

Assignment uses `:=`, and a query result is assigned with `SELECT ... INTO`.

`%TYPE` and `%ROWTYPE` are worth the habit: they tie the variable to the column or table definition, so the code keeps working if the column type changes.



An anonymous block: DO

You do not have to create a routine to run PL/pgSQL. A `DO` block executes it once, immediately — ideal for one-off maintenance scripts:

```
DO $$
DECLARE
    cutoff          constant int := 80;
    hits            integer;
    total           integer;
    oldest          tracks%ROWTYPE;
BEGIN
    SELECT count(*) INTO hits FROM tracks WHERE popularity >= cutoff;
    SELECT count(*) INTO total FROM tracks;

    SELECT * INTO oldest FROM tracks ORDER BY release_date LIMIT 1;

    -- '%' is the placeholder, so a literal percent sign has to be doubled.
    -- Writing them adjacent is ambiguous, so keep the word instead.
    RAISE NOTICE 'Tracks at popularity >= %: % of % (% percent)',
        cutoff, hits, total, round(100.0 * hits / total, 2);
    RAISE NOTICE 'Oldest track: "%" from %', oldest.name, oldest.release_date;
END;
$;
```



2.4 VARIABLES

An anonymous block: DO

A **DO** block takes no parameters and returns nothing — it acts. Its output goes to the notice channel, which the playground displays above the results.

Looping over a query, with a variable accumulating as it goes:

```
DO $$
DECLARE
    rec          record;
    running_total integer := 0;
BEGIN
    FOR rec IN
        SELECT decade_of(release_date) AS decade, count(*) AS tracks
        FROM tracks
        GROUP BY 1
        ORDER BY 1
    LOOP
        running_total := running_total + rec.tracks;
        RAISE NOTICE '% → % tracks (running total %)',
            rec.decade, rec.tracks, running_total;
    END LOOP;
END;
$$;
```



An anonymous block: DO

💡 Everything that loop does, `SUM(...) OVER (ORDER BY decade)` from Lecture 1 §1.3 does in one statement and far faster. Reach for the window function first.



2. Session variables

These live in the connection, not in a block, and are visible to every statement until the session ends.

```
SET my.decade = '1970';
```

```
SELECT current_setting('my.decade') AS the_setting;
```

RESULT

the_setting
1970



2. Session variables

```
SELECT count(*) AS tracks_in_my_decade
FROM tracks
WHERE decade_of(release_date) = current_setting('my.decade')::int;
```

RESULT

tracks_in_my_decade
6325



2. Session variables

Note the cast: `current_setting` always returns `text`, whatever you put in.

`SET LOCAL` limits the setting to the current transaction:

```
BEGIN;  
SET LOCAL my.decade = '1990';  
SELECT current_setting('my.decade') AS inside_transaction;  
COMMIT;  
  
SELECT current_setting('my.decade') AS after_commit;
```

RESULT

after_commit
1970



2.4 VARIABLES

2. Session variables

The same mechanism configures the server's own behaviour:

```
SET work_mem = '256MB';  
SHOW work_mem;
```

RESULT

work_mem
256MB



A custom session variable must contain a dot (my.decade, not decade) — that is how PostgreSQL distinguishes user settings from its own configuration parameters.



3. psql client variables

Strictly speaking these belong to the `psql` client rather than to the server, so they will not run in the browser playground — but they are what people usually reach for in scripts:

```
\set decade 1970  
SELECT count(*) FROM tracks WHERE decade_of(release_date) = :decade;
```

`:name` substitutes the value, `:'name'` substitutes it quoted as a string literal, and `:"name"` as an identifier.



Which one to use

Need	Use
A temporary value inside a routine or block	a PL/pgSQL variable in <code>DECLARE</code>
A value shared by several statements in one connection	<code>SET</code> / <code>current_setting()</code>
A parameter for a script run through the <code>psql</code> client	<code>\set</code> and <code>:name</code>
A value passed from an application	a query parameter (<code>\$1</code> , <code>\$2</code>), never string concatenation

The last row is a security point, not a style point: building SQL by concatenating user input is how SQL injection happens. Pass parameters.



Which one to use

-- never do this

```
'SELECT * FROM tracks WHERE name = '' || user_input || '''
```

-- do this: the value can never be parsed as SQL

```
SELECT * FROM tracks WHERE name = $1
```



Summary

DDL describes the structure of the data — tables, indexes, constraints, sequences:

- **CREATE TABLE** — creates a table; declare **PRIMARY KEY**, **NOT NULL**, **UNIQUE**, **CHECK**, **REFERENCES** and **DEFAULT** while you are there;
- **ALTER TABLE** — renames a table, and adds, renames, retypes and drops fields;
- **DROP COLUMN / DROP TABLE** — removes a field or the whole table;
- **TRUNCATE TABLE** — removes all records fast, keeping the structure;
- **CREATE TABLE ... AS SELECT** — creates and fills a table from a query in one go.



Summary

DML manipulates the data in the tables:

- **INSERT INTO** — adds records, from **VALUES** or from a **SELECT**; **ON CONFLICT** turns it into an upsert;
- **UPDATE** — changes data, with or without a filter; **UPDATE ... FROM** draws new values from another table;
- **DELETE** — deletes data; always give it a condition, and use **TRUNCATE** when you mean "everything";
- **RETURNING** reports what any of the three actually changed.

Materializations name a query:

- a **view** stores only the query — it is always current, occupies no space, and costs a full re-execution on every read;
- a **materialized view** stores the result — reads are cheap and it can be indexed, but the data is only as fresh as the last **REFRESH MATERIALIZED VIEW**;
- unlike a CTE, both persist in the database and can be used by other queries.



Summary

Procedures and functions put logic in the database, written in PL/pgSQL:

- a **FUNCTION** returns a value and can be called inside a query (**SELECT f(x)**);
- a **PROCEDURE** performs an action, is called with **CALL**, and can manage transactions;
- both support **IN/OUT/INOUT** parameters, **IF**, **FOR/WHILE** loops, and **EXCEPTION** blocks;
- prefer set-based statements to row-by-row loops.

Variables come in three kinds, at three different scopes:

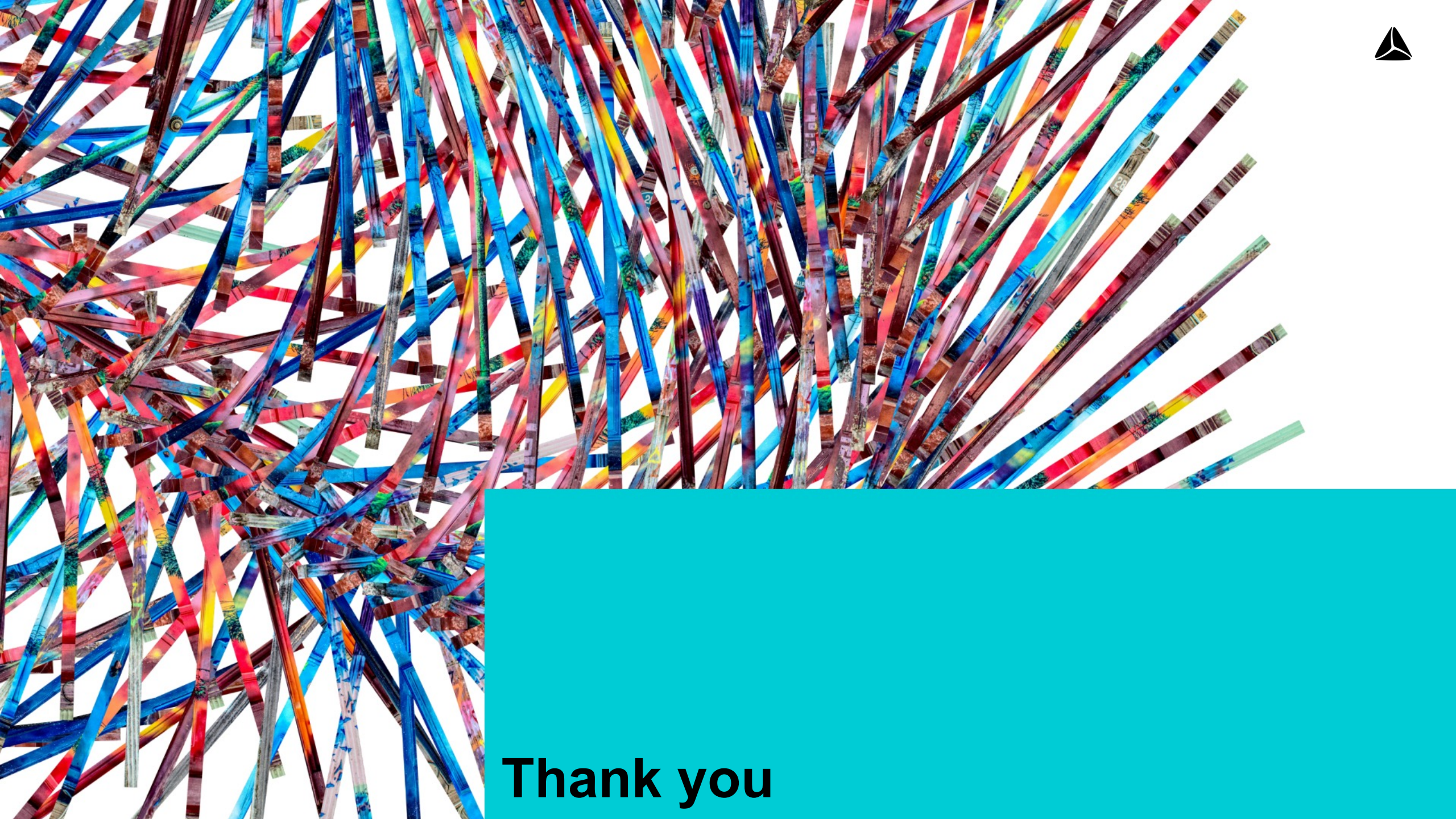
- PL/pgSQL variables — declared in **DECLARE**, assigned with **:=** or **SELECT ... INTO**, alive for one call; **%TYPE** and **%ROWTYPE** tie them to the schema;
- session variables — **SET / SET LOCAL**, read with **current_setting()**, alive for the connection or the transaction;
- psql client variables — **\set** and **:name**, for scripts.

Values coming from an application belong in query parameters, never in concatenated SQL.



TBC Explorers

Q & A



Thank you